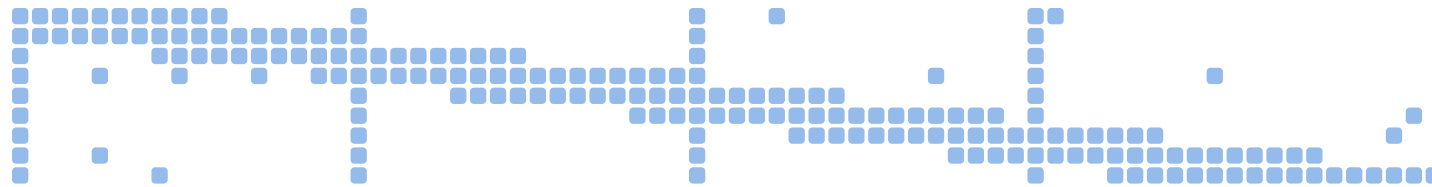


APPT 2026 · STUDENT GPU PROGRAMMING CHALLENGE

Structure-Aware SpGEMM on GPUs



A portable hash engine and a **zero-profiling dispatcher**
for $C = A \cdot A$ and $C = A \cdot A^T$ over the SuiteSparse collection

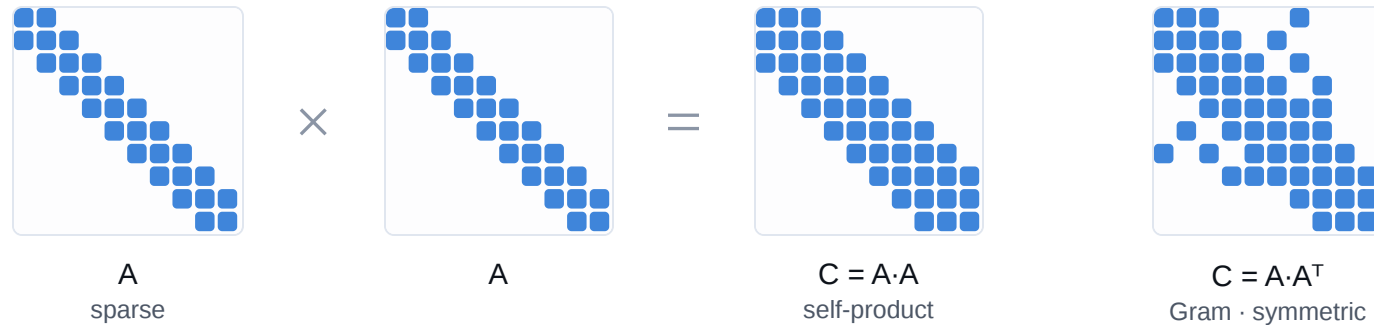
100 real SuiteSparse matrices

target: single A100 80 GB

measured on H100, sm_80-only build

BACKGROUND

Two sparse products — the cost is *irregularity*, not arithmetic



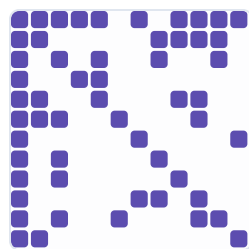
The output size is **unknown in advance** · row lengths **vary by orders of magnitude** · every memory access is **data-dependent**

THE HARD PART

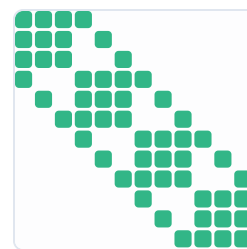
Real matrices are heterogeneous — no single kernel wins



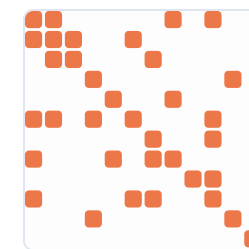
banded PDE



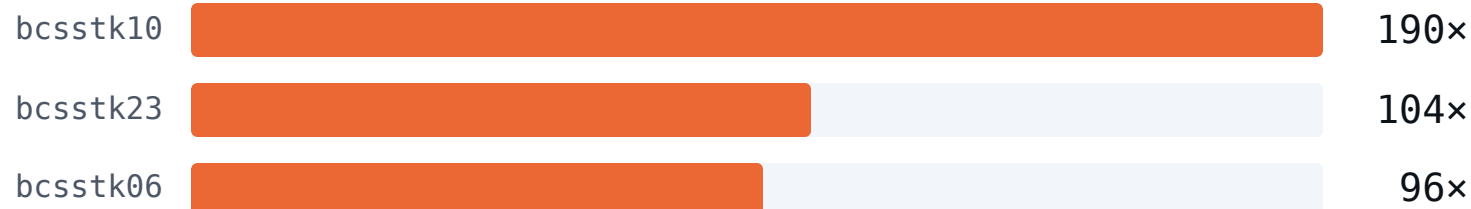
power-law graph



stiffness matrix

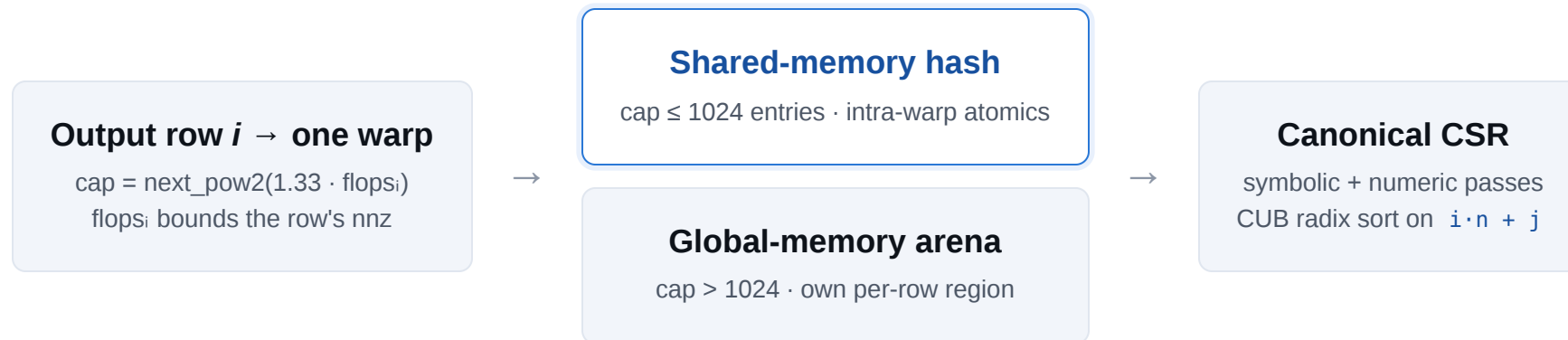


circuit · unsymmetric



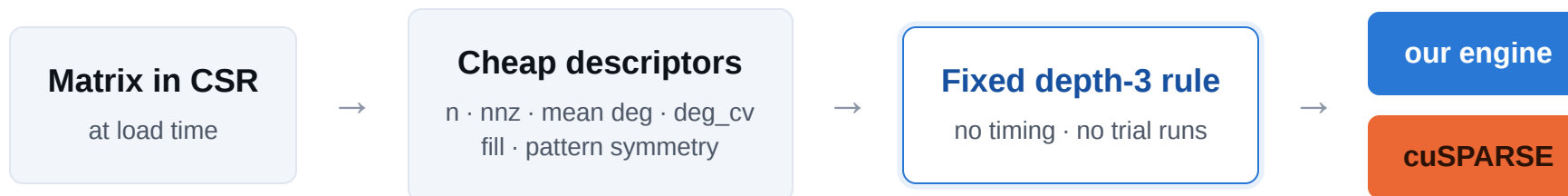
Even the vendor library falls into **pathological paths**: cuSPARSE is up to **190x slower** than our simple engine on structured stiffness matrices

A portable hash engine — one warp per row, sized per row

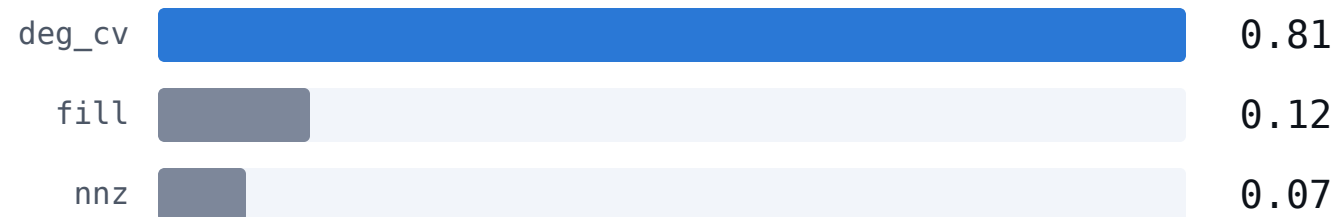


Open addressing · linear probing · pure `atomicCAS` insert · one templated `A·B` engine for both tasks (`B = A` or `B = AT`, transpose timed in) ·
sm_80 only · byte-exact against cuSPARSE on all 258 tasks

A zero-profiling dispatcher

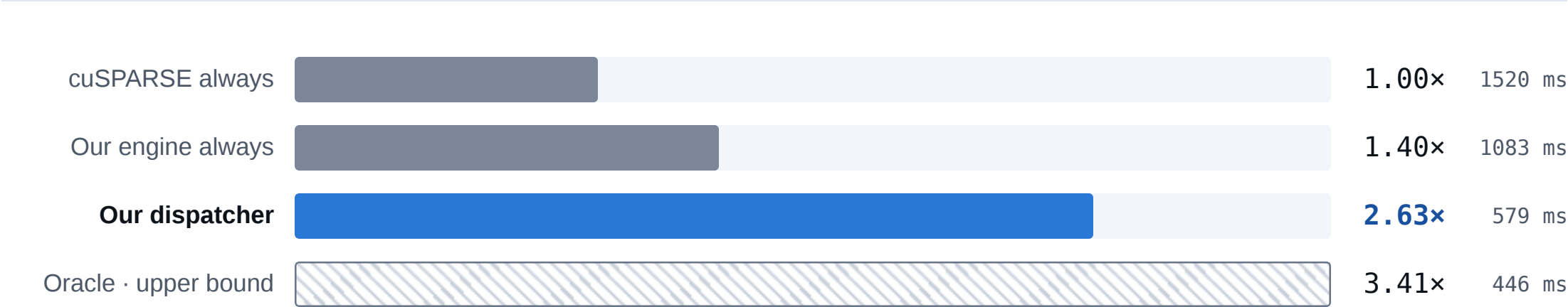


use ours iff $(\text{deg_cv} \leq 0.63 \wedge \text{fill} \leq 53) \vee (\text{deg_cv} > 0.63 \wedge \text{nnz} > 20\text{k} \wedge \text{fill} \leq 47)$



Decision-feature importance — dominated by **row-length imbalance (deg_cv)**, the very irregularity that makes SpGEMM hard

Speedup over cuSPARSE

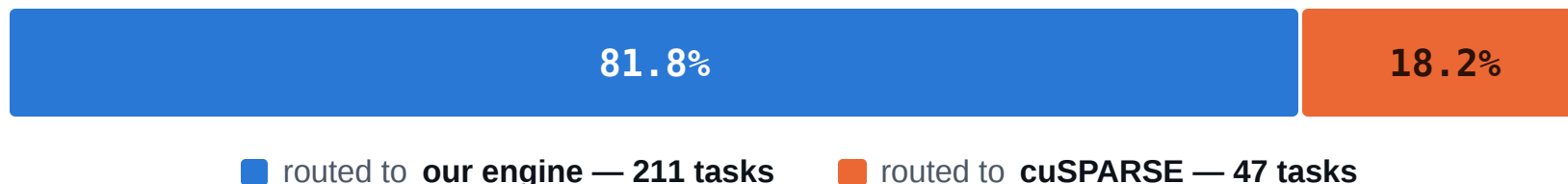


2.63× over cuSPARSE — the dispatcher removes **88% of the time the oracle could remove** (1520 → 579 ms, against 446 ms) · **0 / 258** correctness mismatches

Descriptor cost: <1% on the small-matrix regime, 8.6% aggregated over the suite (a host 0 (nnz) pass on the largest matrices — movable to the GPU).
Measured on H100 with sm_80-only binaries; cuSPARSE keeps its Hopper paths there, so these ratios should hold or improve on the A100.

EFFECT · ROBUSTNESS BY CONSTRUCTION

Route each matrix to whatever solves it fastest



On the official small-matrix regime alone, our engine solves the suite **2.66× faster in aggregate** (387 vs 1029 ms) and wins **78–82%** of matrices. It loses on a few very large, high-fill ones — and that is **exactly where the dispatcher routes the work away**.

The system never depends on one kernel being universally best.

TAKEAWAY

Real-world SpGEMM is a *generality* problem

A correct, portable hash engine plus a **profiling-free, structure-aware dispatcher** — **2.63× over cuSPARSE**, driven by row-length imbalance.
Route each matrix to whatever solves it fastest.

Thank you — questions welcome.